

Seguridad y resiliencia

Seguridad y resiliencia del sistema

Objetivo

El ecosistema fue diseñado para que la automatización pueda operar de forma autónoma sin perder control sobre situaciones sensibles.

Para ello se incorporaron tres principios principales:

- **minimización de datos;**
- **gestión de errores y trazabilidad;**
- **Human-in-the-loop antes de acciones críticas.**

El objetivo es evitar respuestas incorrectas, pérdida silenciosa de información y acciones automáticas que requieran supervisión humana.

Minimización de datos

El flujo procesa únicamente los datos necesarios para atender la consulta.

Desde Gmail se utilizan principalmente:

Nombre
Email
Asunto
Mensaje
Message ID
Thread ID
Fecha

No se almacenan adjuntos, credenciales, contraseñas ni información ajena al objetivo del proceso.

Asimismo, el modelo de IA no recibe la base completa de Airtable.

Primero se clasifica la intención y luego se recupera únicamente el conocimiento relacionado con esa categoría.

Consulta
↓
Clasificación
↓
Categoría relevante
↓
Información necesaria de Airtable
↓
GPT

Esto disminuye tanto la exposición innecesaria de datos como el consumo de tokens.

Separación entre datos y conocimiento

La arquitectura separa la información operativa de la información utilizada para responder.

La tabla **Consultas** contiene los datos correspondientes a cada interacción con el cliente.

La tabla **Conocimiento** contiene exclusivamente información oficial del gimnasio.

El modelo generativo recibe instrucciones para utilizar únicamente la información recuperada desde esta base de conocimiento y no completar respuestas utilizando información externa o inventada.

De esta forma se reduce el riesgo de alucinaciones.

Validación de datos de entrada

Antes de procesar una consulta, el workflow ejecuta el nodo:

IF - Validar datos de entrada

El sistema verifica que existan como mínimo:

From ≠ vacío
AND
snippet ≠ vacío

Si ambas condiciones son correctas, el proceso continúa.

Si alguno de estos datos no está disponible, la consulta no se envía al modelo de IA.

En su lugar se activa la ruta de error:

Datos incompletos

↓

LOG ERROR - Datos incompletos

El registro almacena:

```
{  
  "Resultado": "ERROR",  
  "Nodo": "Validación de entrada",  
  "Detalle Error": "Datos incompletos: falta remitente o mensaje.",  
  "Requirió HITL": false,  
  "Aprobado": false  
}
```

De esta forma se evita gastar recursos de IA sobre entradas inválidas.

Protección contra bucles infinitos

El Gmail Trigger incorpora un filtro para evitar que los correos enviados por la propia automatización vuelvan a activar el workflow.

La condición utilizada es:

in:inbox -from:luкас.tarantostevanovich@gmail.com

Esto permite que el sistema procese mensajes recibidos en Inbox, pero excluya los enviados desde la cuenta utilizada por el propio flujo.

Sin este control podría producirse la siguiente situación:

Correo recibido

↓

n8n responde

↓

n8n detecta su propia respuesta

↓

vuelve a responder

↓

loop infinito

Con el filtro implementado:

Correo externo

↓

Workflow

↓

Respuesta automática

↓

Filtro excluye remitente propio

↓

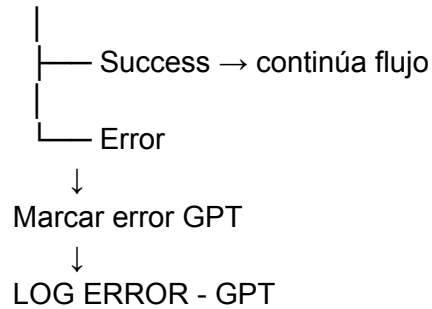
FIN

Manejo de errores del motor de IA

El nodo de clasificación GPT posee una salida específica para errores.

En lugar de detener completamente el workflow ante una falla de OpenAI, el sistema utiliza una ruta alternativa de error.

GPT - Clasificar intención



En caso de falla, el registro de **Consultas** se actualiza con:

Estado = Error

Confianza = 0

Error = Error en GPT - Clasificación

Además, se crea un registro de auditoría en **Log Ejecuciones**.

Ejemplo:

```
{
  "Resultado": "ERROR",
  "Nodo": "GPT - Clasificar y responder",
  "Detalle Error": "Error en GPT - Clasificación",
  "Requirió HITL": false,
  "Aprobado": false
}
```

Esto permite identificar posteriormente qué consultas fallaron y en qué etapa.

Estrategia de Error Handling

La gestión de errores implementada sigue el principio de **continuar mediante una salida controlada de error**, en lugar de permitir que una falla externa produzca una ejecución incompleta sin registro.

La configuración del nodo de IA permite continuar utilizando su salida de error.

Conceptualmente:

API disponible
→ ejecución normal

API con error
→ salida de error
→ actualizar Airtable
→ registrar Log
→ finalizar de forma controlada

Esta estrategia aporta resiliencia porque una falla externa no elimina la trazabilidad de la consulta.

Human-in-the-loop

El sistema incluye un mecanismo de **Human-in-the-loop (HITL)** para las consultas consideradas sensibles.

Las categorías definidas como críticas son:

BAJA
RECLAMO
QUEJA
COBRO
OTRO

Estas categorías siempre generan:

```
{  
  "requiere_humano": true  
}
```

También pueden requerir revisión humana las consultas cuya intención no pueda determinarse con suficiente seguridad.

En estos casos se utiliza:

OTRO

+

requiere_humano = true

Flujo HITL

Cuando el sistema determina que una consulta requiere intervención humana, no envía directamente la respuesta al cliente.

La secuencia es:

GPT redacta propuesta

↓

¿Requiere humano?

↓ Sí

Gmail - Solicitar aprobación HITL

↓

Workflow espera respuesta humana

↓

¿Aprobado por humano?

Existen dos resultados posibles.

Aprobación

Approve

↓

Enviar respuesta aprobada

↓

Estado = Respondido

↓

Aprobado = true

↓

LOG - HITL aprobado

Rechazo

Decline

↓

No se envía respuesta automática

↓

Estado = Revisión manual

↓

Aprobado = false

↓

LOG - HITL rechazado

Esto evita el denominado “**efecto metralleta**”, donde una IA podría ejecutar automáticamente acciones sensibles sin validación previa.

Casos críticos protegidos

Por ejemplo, ante:

"Me cobraron dos veces la cuota."

el sistema puede clasificar:

```
{  
  "intencion": "COBRO",  
  "confianza": 0.95,  
  "requiere_humano": true,  
  "tema_conocimiento": "Cobros y reclamos"  
}
```

La IA puede preparar una respuesta, pero no puede enviarla directamente.

Antes debe existir una aprobación humana.

Lo mismo ocurre con una solicitud como:

"Quiero darme de baja."

La IA no puede confirmar una baja de forma autónoma.

Su función se limita a:

Interpretar

↓

Consultar conocimiento

↓

Proponer respuesta

↓

Solicitar aprobación

Control de tipos de datos

El workflow realiza conversiones explícitas para evitar errores derivados de comparar tipos incompatibles.

Por ejemplo:

Confianza → Number
Requiere Humano → Boolean
Aprobado → Boolean

Esto es especialmente importante en los nodos IF.

Ejemplo:

```
{{ $('Parsear clasificación GPT').first().json.requiere_humano.toBoolean() }}
```

De esta forma se evita comparar:

"false"

como texto contra:

false

como booleano.

La conversión explícita reduce errores de lógica dentro del flujo.

Prompt dinámico y controlado

Los prompts no contienen datos fijos del cliente.

Utilizan variables recibidas durante cada ejecución.

Por ejemplo:

MENSAJE DEL CLIENTE:

```
{{ $('Gmail - Trigger consultas').item.json.snippet }}
```

También se incorporan dinámicamente:

Nombre
Consulta
Información de Airtable
Restricciones

Esto permite reutilizar el mismo workflow con diferentes consultas sin modificar manualmente el prompt.

Protección frente a información no autorizada

La segunda llamada a GPT incluye instrucciones explícitas:

Responder exclusivamente con la información proporcionada por la base de conocimiento.

No inventar datos.

No agregar información externa.

Si la información disponible no alcanza, indicar que un asesor deberá revisar la consulta.

Esto constituye una barrera adicional frente a alucinaciones o respuestas basadas en datos no controlados.

Conservación del hilo de Gmail

Las respuestas al cliente utilizan la operación:

Gmail → Message → Reply

sobre el **Message ID** original.

El sistema también conserva el **Gmail Thread ID** dentro de Airtable.

Esto permite:

- mantener organizada la conversación;
 - evitar generar correos aislados;
 - conservar trazabilidad entre consulta y respuesta;
 - facilitar una posterior revisión humana.
-

Auditoría y trazabilidad

Todas las rutas principales generan información persistente en Airtable.

El sistema permite diferenciar:

OK
ERROR
REVISIÓN MANUAL

También registra:

Consulta asociada
Fecha
Nodo
Detalle del error
Requirió HITL
Aprobado

Esta información alimenta posteriormente el Dashboard de Control.

Por lo tanto, ante una incidencia es posible determinar:

Qué consulta ocurrió
↓
Qué decisión tomó la IA
↓
Si intervino una persona
↓
Si se aprobó o rechazó
↓
Si ocurrió un error
↓
En qué etapa ocurrió

Pruebas de resiliencia realizadas

Durante el desarrollo se probaron distintos escenarios para validar las rutas del sistema.

Caso probado	Resultado esperado
Consulta simple de horarios	Respuesta automática
Consulta sensible de cobro	HITL
HITL aprobado	Respuesta al cliente
HITL rechazado	Revisión manual
Datos incompletos	Registro ERROR
Falla del modelo GPT	Estado Error + Log
Correo enviado por la propia automatización	Excluido por filtro anti-loop

Estas pruebas permiten verificar tanto el camino feliz como los diferentes **unhappy paths** del sistema.

Principio de fallo seguro

Ante una situación no reconocida, el sistema prioriza no ejecutar automáticamente una acción sensible.

La lógica general es:

Si existe certeza + acción segura
→ Automatizar

Si existe riesgo, ambigüedad o acción sensible
→ Human-in-the-loop

Si existe error técnico
→ Registrar error y detener de forma controlada

Este criterio permite mantener un equilibrio entre autonomía y supervisión.

Conclusión

La arquitectura incorpora seguridad y resiliencia desde el diseño y no únicamente como controles posteriores.

Las principales medidas implementadas son:

- ✓ Minimización de datos
- ✓ Base de conocimiento controlada
- ✓ Validación de entrada
- ✓ Filtro anti-bucle
- ✓ Tipos de datos explícitos
- ✓ Error Handler para GPT
- ✓ Registro persistente de errores
- ✓ Human-in-the-loop
- ✓ Aprobación previa a acciones sensibles
- ✓ Trazabilidad mediante logs
- ✓ Conservación de hilos de Gmail
- ✓ Prompts dinámicos

De esta forma, el ecosistema puede operar con un alto nivel de autonomía sin perder control, trazabilidad ni capacidad de recuperación ante errores.